

DATA 607 - Unit 2b - Prediction

Marc Campo

2026-09-13

Approach

I initially had downloaded a data set that included the variables, not the probabilities and model results. So my approach was initially to run a full logistic model and then work from the saved probabilities.

I have the correct data now, and will work directly from modeling results rather than having to run my own model.

Set Options

```
knitr::opts_chunk$set(echo = TRUE)

install.packages("tinytex", repos = "https://cloud.r-project.org")

## Warning: unable to access index for repository https://cloud.r-project.org/src/contrib:
## cannot open URL 'https://cloud.r-project.org/src/contrib/PACKAGES'

## Warning: package 'tinytex' is not available for this version of R
##
## A version of this package for your version of R might be available elsewhere,
## see the ideas at
## https://cran.r-project.org/doc/manuals/r-patched/R-admin.html#Installing-packages
tinytex::install_tinytex(force=TRUE)

library(ggplot2)
library(tidyverse)

## Warning: package 'tidyverse' was built under R version 4.5.2

library(caret)
library(gt)
library(dplyr)
```

Code Base

Load Penguin Data Set

I uploaded the CSV for the Penguin data set to GitHub.

```
library(dplyr)
options(scipen=999)
data <- read.csv("https://raw.githubusercontent.com/cam5671/DATA-607/main/Unit2/penguins.csv")
```

```
data <- data %>%
  rename(
    Probability = .pred_female,
    Predicted = .pred_class,
    Sex = sex
  )
head(data)
```

```
##   Probability Predicted   Sex
## 1    0.9921746   female female
## 2    0.9542394   female female
## 3    0.9847350   female female
## 4    0.1870206     male female
## 5    0.9947012   female female
## 6    0.9999891   female female
```

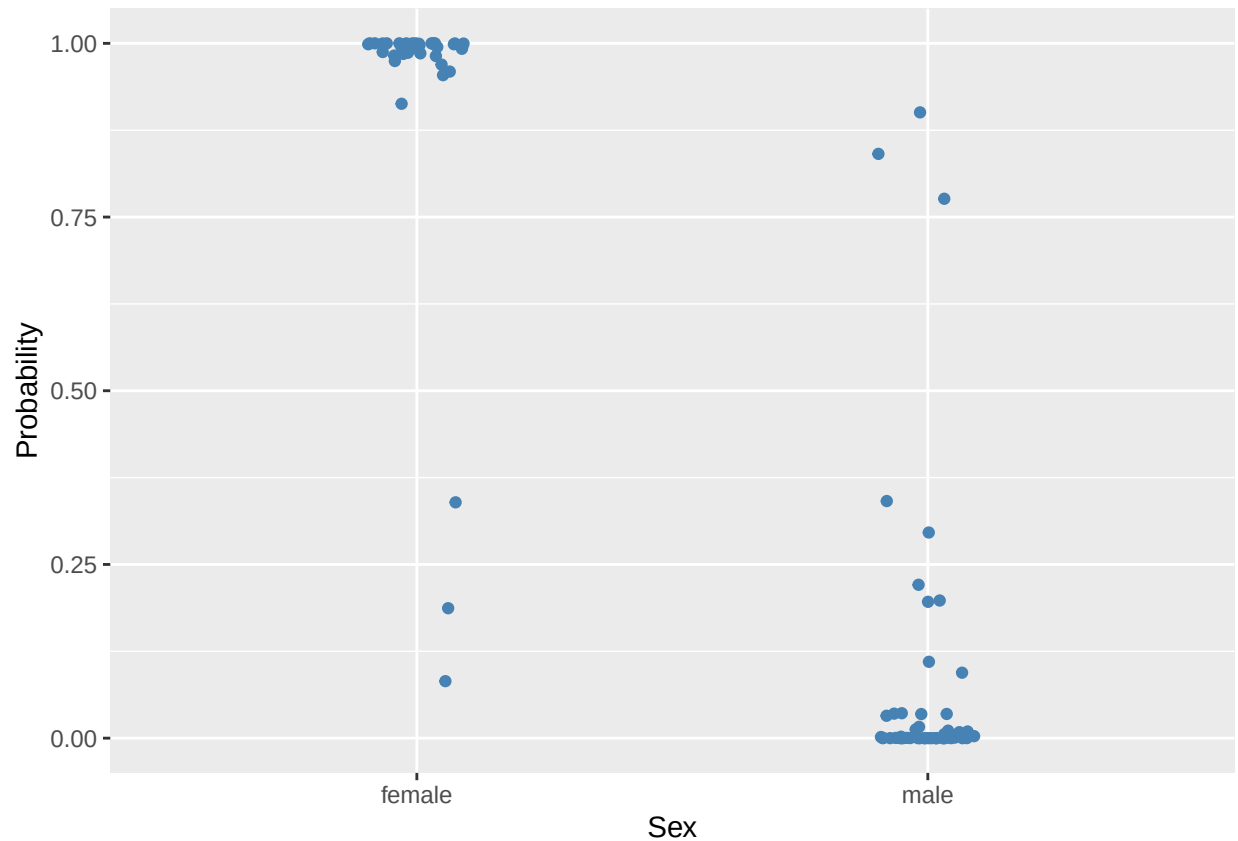
Screen data

The data include the predicted probability of female sex, the predicted sex (based on a .5 threshold), and the actual.

We can chart the probabilities versus the actual sex to see how well they perform. We see that with a few exceptions that the model predicts very well. Females have a very high probability of being female and males have a very low probability.

We can also cross-tabulate the prediction versus actual and we see that there are only 3 wrong predictions for each sex.

```
library(ggplot2)
ggplot(data, aes(y=Probability, x=Sex)) + geom_jitter(width=.1, color='steelblue')
```



```
table(data$Predicted, data$Sex)
```

```
##
##      female male
## female     36   3
## male       3  51
```

Null Error Rate

The null error rate, for these data would be the rate of errors if the model always predicted the more common outcome - Male. The null error rate is simply 1-female classification.

The NER is 42%.

```
female_percent <- sum(data$Sex=="female")/nrow(data)
```

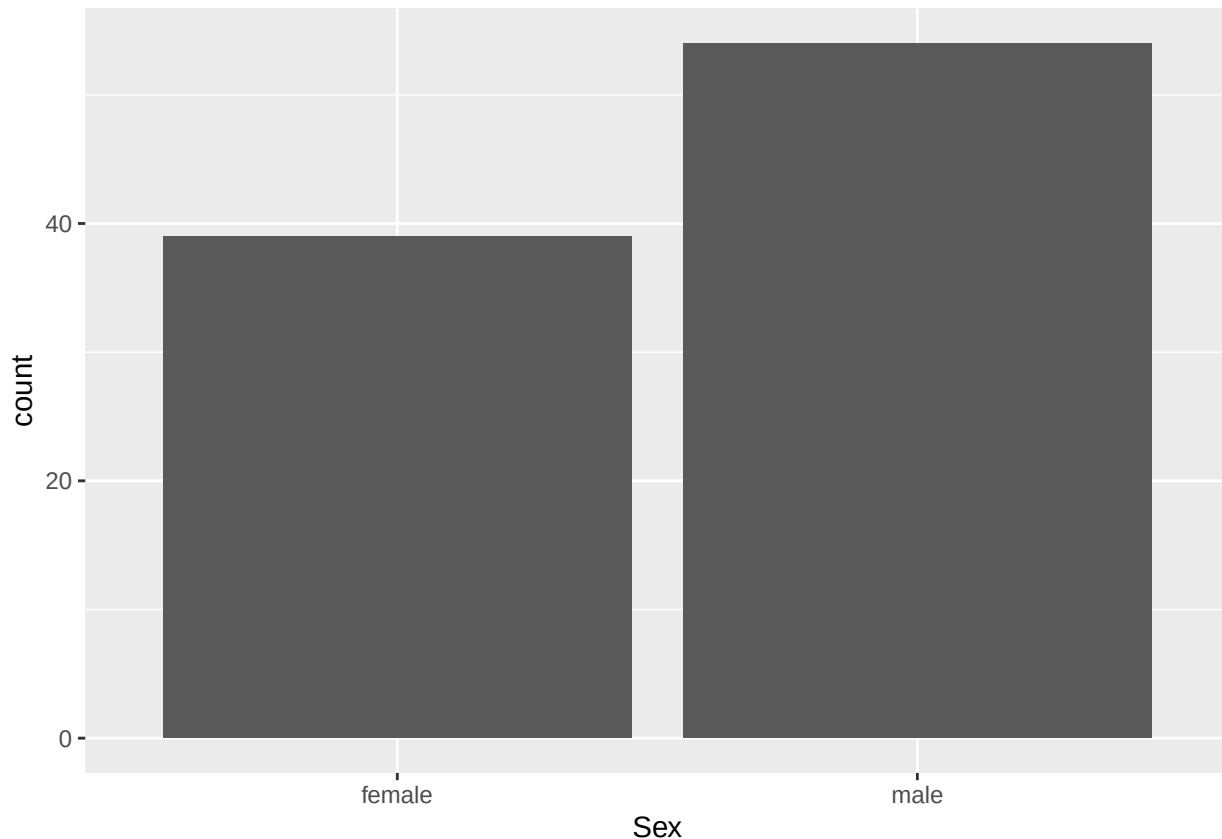
```
print(paste("Null Error Rate = ", female_percent))
```

```
## [1] "Null Error Rate = 0.419354838709677"
```

Actual Distribution

The plot shows a majority of male penguins.

```
ggplot(data, aes(x=Sex)) + geom_bar()
```



Why the NER is important

The null error rate is very important in data analytics. On a very basic level it tells you how much more accurate your predictions were just compared to picking the majority. This is more important than one might think initially. Many modeling techniques, like logistic regression, often predict most cases to be the more common outcome. This leads to poor predictions. Hosmer and Lemeshow¹ admit this in their textbook *Applied Logistic Regression*. We have been able to use techniques like LR for evaluation of predictor importance and magnitude, but not for predicting actual outcomes. Accuracy like we see, above, for the Penguin data set is unheard of in medical research using classic techniques. Machine learning techniques perform more effectively.

Confusion Matrices

We can calculate the confusion matrices manually but the caret package makes it easier. The results are in the table below. All of the thresholds perform very well. As we move the threshold down (making it harder to get a negative prediction) we reduce false negatives and as we move it up (making it harder to get a positive prediction) we reduce false positives which makes sense. The trade off in accuracy, however, is not large. And if we chose our thresholds based on results we have already calculated, you run the risk of constantly tuning your predictions to match existing data which may differ slightly from new data that you want to use your prediction methods for (overfit).

```
library(dplyr)
library(gt)

# Actual outcome: Female = 1, Male = 0
data$Actual <- ifelse(data$Sex == "female", 1, 0)
```

```

# Predictions at each threshold
data$Predicted_2 <- ifelse(data$Probability >= .2, 1, 0)
data$Predicted_5 <- ifelse(data$Probability >= .5, 1, 0)
data$Predicted_8 <- ifelse(data$Probability >= .8, 1, 0)

model_metrics <- function(actual, predicted) {

  TP <- sum(actual == 1 & predicted == 1)
  FP <- sum(actual == 0 & predicted == 1)
  TN <- sum(actual == 0 & predicted == 0)
  FN <- sum(actual == 1 & predicted == 0)

  Accuracy <- (TP + TN) / (TP + TN + FP + FN)
  Precision <- TP / (TP + FP)
  Recall <- TP / (TP + FN)
  F1 <- 2 * (Precision * Recall) / (Precision + Recall)

  c(
    TP = TP,
    FP = FP,
    TN = TN,
    FN = FN,
    Accuracy = Accuracy,
    Precision = Precision,
    Recall = Recall,
    F1 = F1
  )
}

m2 <- model_metrics(data$Actual, data$Predicted_2)
m5 <- model_metrics(data$Actual, data$Predicted_5)
m8 <- model_metrics(data$Actual, data$Predicted_8)

results_table <- data.frame(
  Metric = names(m2),
  `0.2` = m2,
  `0.5` = m5,
  `0.8` = m8,
  check.names = FALSE
)

# Beautiful formatted table
results_table %>%
  gt(rowname_col = "Metric") %>%
  fmt_number(
    columns = c(`0.2`, `0.5`, `0.8`),
    rows = Metric %in% c("Accuracy", "Precision", "Recall", "F1"),
    decimals = 3
  ) %>%

```

Classification Performance by Probability Threshold
Female = Positive Class

	Probability Threshold		
	0.2	0.5	0.8
TP	37	36	36
FP	6	3	2
TN	48	51	52
FN	2	3	3
Accuracy	0.914	0.935	0.946
Precision	0.860	0.923	0.947
Recall	0.949	0.923	0.923
F1	0.902	0.923	0.935

```
fmt_number(
  columns = c(`0.2`, `0.5`, `0.8`),
  rows = Metric %in% c("TP", "FP", "TN", "FN"),
  decimals = 0
) %>%
tab_header(
  title = "Classification Performance by Probability Threshold",
  subtitle = "Female = Positive Class"
) %>%
tab_spanner(
  label = "Probability Threshold",
  columns = c(`0.2`, `0.5`, `0.8`)
) %>%
tab_style(
  style = cell_text(weight = "bold"),
  locations = cells_stub(rows = everything())
) %>%
opt_row_stripping()
```

Threshold Use Cases

The choice of threshold depends on the outcome and the real-world implications of your data. A simple way to examine thresholds would be to look at the number and percentages of false positive and false negatives created when you shift the threshold and compare that the actions that will ensue from the predictions.

We see this in medical diagnosis prediction. If a false negative results in overlooking a devastating diagnosis for the patient, we would want to shift the probability lower. On the other hand, if a positive diagnosis results in potentially harmful, invasive testing we would want to limit the number of false positives.

The ACSVD risk model for cardiovascular disease can be used with a variety of probability thresholds (5% to 20%) to decide on treatment.² The recommendation is for clinicians to discuss the risk/reward of interventions for hyperlipidemia (which is only one of the risk factors for CVD).

References

1. Hosmer DW, Lemeshow S, Sturdivant RX. Applied logistic regression. New York: wiley; 2000 Jan.

2. Members WC, Blumenthal RS, Morris PB, Gaudino M, Johnson HM, Anderson TS, Bittner VA, Blankstein R, Brewer LC, Cho L, de Ferranti SD. 2026 ACC/AHA/AACVPR/ABC/ACPM/ADA/AGS/APhA/ASPC/NL guideline on the management of dyslipidemia: a report of the American College of Cardiology/American Heart Association joint committee on clinical practice guidelines. *Circulation*. 2026.